1 Homework 3: ResNet20 on CIFAR-10

1.1 Part I - CIFAR-10 training

[1]:
```
import os
import sys
import json
import math
import random
import time
from pathlib import Path

import numpy as np
import torch
from torch import nn
from torch.utils.data import DataLoader, Subset
from torchvision import datasets, transforms

# Work from software/, hw/, or the repository root.
here = Path.cwd().resolve()
candidates = [here, here.parent, here / 'software', here.parent / 'software']
SOFTWARE = next(p for p in candidates if (p / 'models' / 'resnet.py').is_file())
sys.path.insert(0, str(SOFTWARE))
from models.resnet import resnet20_cifar

MODEL_NAME = 'ResNet20'
SEED = 284
EPOCHS = 100
BATCH_SIZE = 128
LEARNING_RATE = 0.1
MOMENTUM = 0.9
WEIGHT_DECAY = 5e-4
RUN_TRAINING = os.environ.get('HW3_TRAIN', '0') == '1'
DEVICE = torch.device('cuda' if torch.cuda.is_available() else
                      'mps' if torch.backends.mps.is_available() else 'cpu')
torch.set_num_threads(min(8, os.cpu_count() or 1))
random.seed(SEED)
np.random.seed(SEED)
torch.manual_seed(SEED)
if DEVICE.type == 'cuda':
    torch.backends.cudnn.benchmark = True

RESULT = SOFTWARE / 'result' / MODEL_NAME.lower()
RESULT.mkdir(parents=True, exist_ok=True)
CHECKPOINT = RESULT / 'model_best.pth'
CONFIG = dict(model=MODEL_NAME, seed=SEED, epochs=EPOCHS, batch_size=BATCH_SIZE,
              learning_rate=LEARNING_RATE, momentum=MOMENTUM,
              weight_decay=WEIGHT_DECAY, validation_size=5000,
              optimizer='SGD', scheduler='5-epoch warmup, cosine decay to 0.001')
```

1.1.1 CIFAR-10

[2]:
```
normalize = transforms.Normalize((0.491, 0.482, 0.447), (0.247, 0.243, 0.262))
test_transform = transforms.Compose([transforms.ToTensor(), normalize])
test_dataset = datasets.CIFAR10(SOFTWARE / 'data', train=False,
                                download=True, transform=test_transform)
testloader = DataLoader(test_dataset, batch_size=BATCH_SIZE, shuffle=False,
                        num_workers=0)

@torch.inference_mode()
```

```
def validate(model, loader):
    model.eval()
    correct = torch.zeros((), device=DEVICE, dtype=torch.long)
    loss_sum = torch.zeros((), device=DEVICE)
    total = 0
    for images, labels in loader:
        images = images.to(DEVICE)
        labels = labels.to(DEVICE)
        logits = model(images)
        loss_sum += nn.functional.cross_entropy(logits, labels, reduction='sum')
        correct += (logits.argmax(1) == labels).sum()
        total += labels.numel()
    return dict(correct=int(correct.item()), total=total,
                accuracy=100.0 * correct.item() / total,
                loss=loss_sum.item() / total)
```

1.1.2 Training

[3]:
```
def save_checkpoint(state, destination):
    temporary = destination.with_suffix('.tmp')
    torch.save(state, temporary)
    temporary.replace(destination)

if RUN_TRAINING:
    augmentation = transforms.Compose([
        transforms.RandomCrop(32, padding=4),
        transforms.RandomHorizontalFlip(), transforms.ToTensor(), normalize,
    ])
    training = datasets.CIFAR10(SOFTWARE / 'data', train=True,
                                download=True, transform=augmentation)
    validation = datasets.CIFAR10(SOFTWARE / 'data', train=True,
                                  download=False, transform=test_transform)
    indices = torch.randperm(50000, generator=torch.Generator().manual_seed(SEED))
    training = Subset(training, indices[5000:].tolist())
    validation = Subset(validation, indices[:5000].tolist())
    generator = torch.Generator()
    trainloader = DataLoader(training, batch_size=BATCH_SIZE, shuffle=True,
                             num_workers=0, generator=generator)
    validationloader = DataLoader(validation, batch_size=BATCH_SIZE,
                                  shuffle=False, num_workers=0)
    model = resnet20_cifar().to(DEVICE)
    optimizer = torch.optim.SGD(model.parameters(), lr=LEARNING_RATE,
                                momentum=MOMENTUM, weight_decay=WEIGHT_DECAY)
    latest = RESULT / 'checkpoint.pth'
    first_epoch, best_validation = 0, -1.0
    if latest.exists():
        saved = torch.load(latest, map_location='cpu', weights_only=True)
        assert saved['config'] == CONFIG, 'Configuration differs from saved run.'
        model.load_state_dict(saved['state_dict'])
        optimizer.load_state_dict(saved['optimizer'])
        first_epoch = saved['epoch']
        best_validation = saved['best_validation']

    for epoch in range(first_epoch, EPOCHS):
        # Epoch seeds also make a resumed run reproducible with num_workers=0.
        torch.manual_seed(SEED + epoch)
        random.seed(SEED + epoch)
        np.random.seed(SEED + epoch)
        generator.manual_seed(SEED + epoch)
        if epoch < 5:
```

```
            lr = LEARNING_RATE * (epoch + 1) / 5
        else:
            phase = (epoch - 5) / (EPOCHS - 6)
            lr = 0.001 + 0.5 * (LEARNING_RATE - 0.001) * (1 + math.cos(math.pi * phase))
        for group in optimizer.param_groups:
            group['lr'] = lr
        started = time.monotonic()
        model.train()
        for images, labels in trainloader:
            images = images.to(DEVICE)
            labels = labels.to(DEVICE)
            optimizer.zero_grad(set_to_none=True)
            loss = nn.functional.cross_entropy(model(images), labels)
            loss.backward()
            optimizer.step()
        if not torch.isfinite(loss).item():
            raise RuntimeError('Non-finite training loss')
        validation_result = validate(model, validationloader)
        improved = validation_result['accuracy'] > best_validation
        best_validation = max(best_validation, validation_result['accuracy'])
        state = dict(state_dict=model.state_dict(), config=CONFIG,
                     epoch=epoch + 1, best_validation=best_validation,
                     validation=validation_result)
        if improved:
            save_checkpoint(state, CHECKPOINT)
        state['optimizer'] = optimizer.state_dict()
        save_checkpoint(state, latest)
        record = dict(epoch=epoch + 1, lr=lr, validation=validation_result,
                      best_validation=best_validation,
                      seconds=time.monotonic() - started)
        with (RESULT / 'training.jsonl').open('a') as log:
            log.write(json.dumps(record) + '\n')
    del model, optimizer
    if DEVICE.type == 'mps':
        torch.mps.empty_cache()
```

1.1.3 Test accuracy

[4]:
```
if not CHECKPOINT.exists():
    raise FileNotFoundError(f'Train first: missing {CHECKPOINT}')
checkpoint = torch.load(CHECKPOINT, map_location='cpu', weights_only=True)
assert checkpoint['config'] == CONFIG
model = resnet20_cifar().to(DEVICE)
model.load_state_dict(checkpoint['state_dict'], strict=True)
model.eval()
final = validate(model, testloader)
assert final['total'] == 10000
print(f"{MODEL_NAME}: final CIFAR-10 test accuracy = {final['accuracy']:.2f}%")
print(f"Correct: {final['correct']:,}/{final['total']:,}; mean cross-entropy:
 ↪  {final['loss']:.6f}")
print(f"Checkpoint epoch: {checkpoint['epoch']}; selected using validation accuracy.")
assert final['accuracy'] > 90.0, 'The required test accuracy has not been reached.'
```

```
ResNet20: final CIFAR-10 test accuracy = 91.86%
Correct: 9,186/10,000; mean cross-entropy: 0.279553
Checkpoint epoch: 99; selected using validation accuracy.
```

1.2 Part II - Reproduce BasicBlock1's conv1 input

1.2.1 Layer inputs

[5]:
```
model.eval()
captured = {}
def capture_input(name):
    def hook(module, inputs):
        # Keep the input before an in-place ReLU can change it.
        captured[name] = inputs[0].detach().clone()
    return hook

block0 = model.layer1[0]
block1 = model.layer1[1]
assert block0.downsample is None and block0.stride == 1
source_layer = block0.conv1
reference_layer = block1.conv1
handles = [source_layer.register_forward_pre_hook(capture_input('source')),
           reference_layer.register_forward_pre_hook(capture_input('reference'))]
image, label = test_dataset[0]
try:
    with torch.inference_mode():
        logits = model(image.unsqueeze(0).to(DEVICE))
finally:
    for handle in handles:
        handle.remove()
assert set(captured) == {'source', 'reference'}
print(f'Test image index: 0; label: {test_dataset.classes[label]}')
print('Captured source shape:', tuple(captured['source'].shape))
print('Captured reference shape:', tuple(captured['reference'].shape))
```

```
Test image index: 0; label: cat
Captured source shape: (1, 16, 32, 32)
Captured reference shape: (1, 16, 32, 32)
```

1.2.2 Manual calculation

[6]:
```
# The first block keeps 16 channels, so its shortcut is the input itself.
with torch.inference_mode():
    residual = captured['source']
    activation = block0.conv1(captured['source'])
    activation = block0.bn1(activation)
    activation = nn.functional.relu(activation)
    activation = block0.conv2(activation)
    activation = block0.bn2(activation)
    manual = nn.functional.relu(activation + residual)

reference = captured['reference']
difference = (manual - reference).abs()
assert manual.shape == reference.shape
assert torch.isfinite(manual).all().item()
torch.testing.assert_close(manual, reference, rtol=1e-4, atol=1e-5)
print('Manually computed shape:', tuple(manual.shape))
print(f'Maximum absolute difference: {difference.max().item():.8g}')
print(f'Mean absolute difference: {difference.mean().item():.8g}')
print('Match: PASS (rtol=0.0001, atol=0.00001)')
```

```
Manually computed shape: (1, 16, 32, 32)
Maximum absolute difference: 0
Mean absolute difference: 0
Match: PASS (rtol=0.0001, atol=0.00001)
```